Programowanie obiektowe — klasy, obiekty

W jaki sposób możemy reprezentować w programie dane złożone? Jeżeli chcemy np. przechowywać imię, nazwisko i rok urodzenia danej osoby, to możemy przechowywać je w trzech zmiennych:

In [1]:
```
imie = "Jan"
nazwisko = "Kowalski"
rok_urodzenia = 2000
```

Dla każdej osoby, której dane chcemy przechowywać potrzebujemy wtedy trzech zmiennych. Jeżeli chcemy napisać funkcję dostającą informacje o osobie, to funkcja ta musi mieć trzy parametry:

W tych przykładach obliczamy wiek osiągany w podanym roku, bez uwzględniania miesiąca i dnia urodzenia.

In [2]:
```
def wyswietl_wiek(imie, nazwisko, rok_urodzenia, rok):
    print(f"{imie} {nazwisko} ma {rok - rok_urodzenia} lat w roku {rok}.")
```

In [3]:
```
wyswietl_wiek("Jan", "Kowalski", 2000, 2022)
```

```
Jan Kowalski ma 22 lat w roku 2022.
```

Funkcja dostająca informacje o dwóch osobach potrzebuje już co najmniej sześciu parametrów:

In [4]:
```
def wyswietl_porownanie_wieku(
    imie_1,
    nazwisko_1,
    rok_urodzenia_1,
    imie_2,
    nazwisko_2,
    rok_urodzenia_2,
):

    print(f"{imie_1} {nazwisko_1} ", end="")

    if rok_urodzenia_1 < rok_urodzenia_2:
        print("jest starszy(a) niż ", end="")
    elif rok_urodzenia_1 > rok_urodzenia_2:
        print("jest młodszy(a) niż ", end="")
    else:
        print("ma tyle samo lat co", end="")

    print(f"{imie_2} {nazwisko_2}.")
```

In [5]:
```
wyswietl_porownanie_wieku("Jan", "Kowalski", 2000, "Anna", "Nowak", 1995)
```

```
Jan Kowalski jest młodszy(a) niż Anna Nowak.
```

Ten sposób reprezentacji danych nie wyraża w czytelny sposób tego, że trzy dane (imię, nazwisko i rok urodzenia) opisują ten sam obiekt (jedną osobę). Parametr `rok_urodzenia_1` funkcji `wyswietl_porownanie_wieku` nie jest powiązany z parametrem `nazwisko_1` bardziej niż np. z parametrem `nazwisko_2`.

Lepszym pomysłem będzie reprezentacja każdej osoby jako krotki:

In [6]:
```
jan = ("Jan", "Kowalski", 2000)
```

In [7]:
```
anna = ("Anna", "Nowak", 1995)
```

Wtedy powyższe funkcje mogą być zdefiniowane tak:

In [8]:
```
def wyswietl_wiek(osoba, rok):
    print(f"{osoba[0]} {osoba[1]} ma {rok - osoba[2]} lat w roku {rok}.")
```

In [9]:
```
def wyswietl_porownanie_wieku(osoba_1, osoba_2):

    print(f"{osoba_1[0]} {osoba_1[1]} ", end="")

    if osoba_1[2] < osoba_2[2]:
        print("jest starszy(a) niż ", end="")
    elif osoba_1[2] > osoba_2[2]:
        print("jest młodszy(a) niż ", end="")
    else:
        print("ma tyle samo lat co", end="")

    print(f"{osoba_2[0]} {osoba_2[1]}.")
```

Wywołania funkcji stały się czytelniejsze:

In [10]:
```
wyswietl_wiek(jan, 2022)
```

```
Jan Kowalski ma 22 lat w roku 2022.
```

In [11]:
```
wyswietl_porownanie_wieku(jan, anna)
```

```
Jan Kowalski jest młodszy(a) niż Anna Nowak.
```

Niestety stało się to kosztem czytelności kodu funkcji. Nie jest w czytelny sposób wyrażone to, że `osoba[0]` to imię, a `osoba[1]` to nazwisko. Można funkcje napisać nieco czytelniej:

In [12]:
```
def wyswietl_wiek(osoba, rok):
    imie, nazwisko, rok_urodzenia = osoba
    print(f"{imie} {nazwisko} ma {rok - rok_urodzenia} lat w roku {rok}.")
```

In [13]:
```
def wyswietl_porownanie_wieku(osoba_1, osoba_2):

    imie_1, nazwisko_1, rok_urodzenia_1 = osoba_1
    imie_2, nazwisko_2, rok_urodzenia_2 = osoba_2

    print(f"{imie_1} {nazwisko_1} ", end="")

    if rok_urodzenia_1 < rok_urodzenia_2:
        print("jest starszy(a) niż ", end="")
    elif rok_urodzenia_1 > rok_urodzenia_2:
        print("jest młodszy(a) niż ", end="")
    else:
        print("ma tyle samo lat co", end="")

    print(f"{imie_2} {nazwisko_2}.")
```

In [14]:
```
wyswietl_wiek(jan, 2022)
```

```
Jan Kowalski ma 22 lat w roku 2022.
```

Jednak w dalszym ciągu musimy pamiętać, żeby krotka przekazana w argumencie funkcji przechowywała dane w odpowiedniej kolejności (takiej jak oczekuje funkcja).

Co będzie jeżeli przekażemy do funkcji krotki ze złą kolejnością danych?

In [15]:
```
jan = (2000, "Kowalski", "Jan")
```

In [16]:
```
anna = (1995, "Anna", "Kowalska")
```

Błędna kolejność nie zawsze spowoduje błąd. Czasem po prostu da bezsensowne wyniki.

In [17]:
```
wyswietl_porownanie_wieku(jan, anna)
```

```
2000 Kowalski jest starszy(a) niż 1995 Anna.
```

W poniższym przykładzie wystarczy spojrzeć na wyniki, żeby się zorientować w błędzie. Ale czasem błąd może nie być taki oczywisty:

In [18]:
```
george_michael = ("Michael", "George", 1963)
elton_john = ("John", "Elton", 1947)

wyswietl_porownanie_wieku(george_michael, elton_john)
```

```
Michael George jest młodszy(a) niż John Elton.
```

Klasy

Lepszym rozwiązaniem będzie zdefiniowanie własnego typu specjalnie zaprojektowanego do konkretnego celu. W językach obiektowych taki typ nazywa

się **klasą** (ang. *class*).

Uwaga: Ponieważ w Pythonie wszystkie typy (również wbudowane typy proste jak np. `int`) mają cechy, które w wielu innych językach obiektowych dotyczą wyłącznie takich własnych typów, termin „klasa” jest w często kontekście Pythona używany całkowicie zamiennie z terminem „typ”. Natomiast czasem spotyka się również materiały, w których stosowane jest rozróżnienie takie jak w innych językach: na typy (typy wbudowane) i klasy.

Do zdefiniowania nowej klasy służy konstrukcja `class`. Najprostsza możliwa definicja nowej klasy wygląda tak:

In [19]:
```
class Osoba:
    pass
```

Taka definicja jest mało przydatna, ale wystarczy do pokazania podstawowych mechanizmów języka. Dalej definicja zostanie rozwinięta.

Standardową konwencją nazywania klasy jest użycie wielkiej litery na początku każdego wyrazu i małych liter poza tym. Np.

- `DaneOsobowe`,
- `WspolrzedneGeograficzne`.

Dobra praktyka: Standardem jest używanie w kodzie języka angielskiego (np. `PersonalData`, `GeographicCoordinates`) i dobrze się do tego przyzwyczajać ze względu na czytanie dokumentacji (oraz właśnie kodu). W materiałach celowo używany jest jednak język polski, aby ułatwić naukę i skupić się bezpośrednio na istotnych koncepcjach.

Obiekty

Wartość danego typu nazywa się **obiektem** (ang. *object*) lub **egzemplarzem** klasy (ang. class *instance*).

Uwaga: W praktyce dużo częściej od terminu egzemplarz klasy spotyka się termin **instancja** klasy jako kalka z angielskiego instance.

Nowy obiekt klasy możemy stworzyć używając nawiasów po nazwie klasy:

In [20]:
```
jan = Osoba()
```

Obiekt przypisany do zmiennej `jan` jest typu `Osoba`.

Do sprawdzenia typu danej zmiennej lub wartości służy funkcja `type`:

In [21]:
```
type(4)
```

Out[21]:
```
int
```

In [22]:
```
type([1, 2, 3, 4])
```

Out[22]:
```
list
```

In [23]:
```
type(jan)
```

Out[23]:
```
__main__.Osoba
```

In [24]:
```
type(jan) is Osoba
```

Out[24]:
```
True
```

Dobra praktyka: Zalecaną techniką sprawdzania typu jest w większości praktycznych przypadków użycie funkcji `isinstance` a nie `type`. Użycie `isinstance` może w przypadku użycia omawianych w dalszych materiałach technik takich jak *dziedziczenie* dać inne wyniki niż sprawdzenie typu funkcją `type`. Funkcja ta zostanie omówiona w dalszych materiałach dotyczących dziedziczenia.

In [25]:
```
isinstance(jan, Osoba)
```

Out[25]:
```
True
```

In [26]:
```
isinstance(3, int)
```

Out[26]:
```
True
```

Python umożliwia tworzenie typów, których zachowanie jest spójne z zachowaniem typów wbudowanych. Wartości typów wbudowanych mogą być również tworzone w podobny sposób:

In [27]:
```
int()
```

Out[27]:
```
0
```

In [28]:
```
float()
```

Out[28]:
```
0.0
```

In [29]:
```
list()
```

Out[29]:
```
[]
```

In [30]:
```
dict()
```

Out[30]:
```
{}
```

Pola

Obiekt może mieć **pola** (ang. *field*), czyli po prostu przechowywać dane. Do odwołania się do pola używa się tzw. „operatora dostępu do składowej” (pojęcie „składowej” jest dalej wyjaśnione). Pól obiektu używa się tak jak zwykłych zmiennych, czyli np. tworzy się je za pomocą podstawienia:

In [31]:
```
jan.imie = "Jan"
```

In [32]:
```
jan.nazwisko = "Kowalski"
```

In [33]:
```
jan.rok_urodzenia = 2000
```

In [34]:
```
jan.imie
```

Out[34]:
```
'Jan'
```

In [35]:
```
jan.nazwisko
```

Out[35]:
```
'Kowalski'
```

In [36]:
```
jan.rok_urodzenia
```

Out[36]:
```
2000
```

In [37]:
```
jan.rok_urodzenia += 5
```

In [38]:
```
jan.rok_urodzenia
```

Out[38]:
```
2005
```

In [39]:
```
jan.zainteresowania = ["Python", "GIS"]
```

In [40]:
```
jan.zainteresowania
```

Out[40]:
```
['Python', 'GIS']
```

Pola usuwa się również tak jak zwykłe zmienne, czyli za pomocą `del`. Podobnie jak w przypadku zwykłych rzadko jest taka potrzeba i w związku z tym nie jest to często używane:

In [41]:
```
del jan.zainteresowania
```

In [42]:
```
jan.zainteresowania
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[42], line 1
----> 1 jan.zainteresowania

AttributeError: 'Osoba' object has no attribute 'zainteresowania'
```

Teraz funkcje `wyswietl_wiek` i `wyswietl_porownanie_wieku` możemy zdefiniować tak:

In [43]:
```
def wyswietl_wiek(osoba, rok):
    print(
        f"{osoba.imie} {osoba.nazwisko} ma "
        f"{rok - osoba.rok_urodzenia} lat w roku {rok}."
    )
```

In [44]:
```
def wyswietl_porownanie_wieku(osoba_1, osoba_2):
    print(f"{osoba_1.imie} {osoba_1.nazwisko} ", end="")

    if osoba_1.rok_urodzenia < osoba_2.rok_urodzenia:
        print("jest starszy(a) niż ", end="")
    elif osoba_1.rok_urodzenia > osoba_2.rok_urodzenia:
        print("jest młodszy(a) niż ", end="")
    else:
        print("ma tyle samo lat co ", end="")

    print(f"{osoba_2.imie} {osoba_2.nazwisko}.")
```

Zaletą w porównaniu do krotek jest to, że do danych odwołujemy się przez nazwy, a nie przez indeksy.

Wywołania funkcji będą wyglądały tak:

In [45]:
```
wyswietl_wiek(jan, 2022)
```

```
Jan Kowalski ma 17 lat w roku 2022.
```

In [46]:
```
wyswietl_porownanie_wieku(jan, jan)
```

```
Jan Kowalski ma tyle samo lat co Jan Kowalski.
```

Trudniej też teraz przez pomyłkę pomylić kolejność np. imienia z nazwiskiem, bo to do jakiej wartości się odwołujemy jest wyznaczone przez nazwę pola a nie przez kolejność lub indeks.

Konstruktor (metoda `__init__`)

Zazwyczaj pola tworzy się od razu podczas tworzenia obiektu. Żeby to umożliwić klasę można zdefiniować w następujący sposób:

In [47]:
```
class Osoba:
    def __init__(self, imie, nazwisko, rok_urodzenia):
        self.imie = imie
        self.nazwisko = nazwisko
        self.rok_urodzenia = rok_urodzenia
```

Funkcja `__init__` jest wywoływana automatycznie podczas tworzenia obiektu i jest nazywana **konstruktorem** (ang. *constructor*). Jest to przykład **metody** (ang. *method*). Metody są funkcjami powiązanymi z klasą i definiowanymi razem z nią.

Uwaga: Można spotkać się ze stwierdzeniem, że metoda `__init__` nie jest konstruktorem, bo spełnia nieco inną rolę niż konstruktor w wielu innych obiektowych językach programowania. W oficjalnej dokumentacji Pythona metoda `__init__` jest nazywana konstruktorem i taka terminologia jest też używana w tych materiałach.

Poniższa instrukcja tworzy nowy obiekt i podstawia go pod zmienną:

In [48]:
```
jan = Osoba("Jan", "Kowalski", 2000)
```

Instrukcja ta działa w następujący sposób:

- tworzony jest nowy obiekt typu `Osoba`,
- wywoływana jest funkcja `Osoba.__init__` — jako parametr `self` automatycznie przekazywany jest nowo-utworzony obiekt (nie ma go na liście argumentów wywołania), jako pozostałe parametry przekazywane są kolejne argumenty wywołania.
- nowoutworzony obiekt podstawiany jest pod zmienną `jan`.

Obiekt ma teraz utworzone w konstruktorze pola:

In [49]:
```
jan.imie
```

Out[49]:
```
'Jan'
```

In [50]:
```
jan.rok_urodzenia
```

Out[50]:
```
2000
```

Przykład utworzenia kolejnego obiektu:

In [51]:
```
anna = Osoba("Anna", "Nowak", 1950)
```

In [52]:
```
anna.imie
```

Out[52]:
```
'Anna'
```

In [53]:
```
anna.rok_urodzenia
```

Out[53]:
```
1950
```

Sprawdź:

- Wizualizacja działania konstruktora na stronie Python Tutor

Metoda `__init__` jest przykładem tzw. **metody specjalnej** (ang *special method*), tzn. nie jest ona jawnie wywołana, ale jest wywoływana automatycznie w pewnych sytuacjach (w tym przypadku podczas tworzenia obiektu). W języku angielskim spotyka się oprócz określenia „special method” również określenia *magic method* lub *dunder method* (*dunder* jest skrótem od „double underscore” — podwójne podkreślenie).

Nowoutworzony obiekt jest po prostu wartością wyrażenia takiego jak `Osoba("Jan", "Kowalski", 2000)`. Wartość tę możemy po prostu podstawić pod zmienną (jak w przykładzie powyżej), ale możemy też użyć bezpośrednio np. jako argument funkcji lub część większego wyrażenia:

In [54]:
```
wyswietl_porownanie_wieku(
    Osoba("Jan", "Kowalski", 2000), Osoba("Anna", "Nowak", 1990)
)
```

```
Jan Kowalski jest młodszy(a) niż Anna Nowak.
```

In [55]:
```
Osoba("Jan", "Kowalski", 2000).imie
```

Out[55]:
```
'Jan'
```

Metody

Poniższa definicja klasy zawiera również definicję metody `wiek` zwracającej wiek osoby w zadanym roku. Tym razem jest to zwykła metoda a nie metoda specjalna:

In [56]:
```
class Osoba:
    def __init__(self, imie, nazwisko, rok_urodzenia):
        self.imie = imie
        self.nazwisko = nazwisko
        self.rok_urodzenia = rok_urodzenia

    def wiek(self, rok):
        return rok - self.rok_urodzenia
```

Tworzymy obiekt

In [57]:
```
jan = Osoba("Jan", "Kowalski", 2000)
```

i wywołujemy metodę

In [58]:
```
jan.wiek(2015)
```

Out[58]: 15

In [59]:
```
Osoba("Jan", "Kowalski", 2000).wiek(2030)
```

Out[59]: 30

Parametr `self`

Pierwszy parametr (zgodnie z konwencją nazywany zawsze `self`) oznacza zawsze obiekt dla którego wywołujemy metodę. Nie umieszcza się go na liście argumentów wywołania. W wywołaniu jest przed kropką.

Dodajmy jeszcze metodę sprawdzającą czy osoba jest starsza od drugiej osoby:

In [60]:
```
class Osoba:
    def __init__(self, imie, nazwisko, rok_urodzenia):
        self.imie = imie
        self.nazwisko = nazwisko
        self.rok_urodzenia = rok_urodzenia

    def wiek(self, rok):
        return rok - self.rok_urodzenia

    def starszy_od(self, druga_osoba):
        return self.rok_urodzenia < druga_osoba.rok_urodzenia
```

In [61]:
```
jan = Osoba("Jan", "Kowalski", 2000)
```

In [62]:
```
anna = Osoba("Anna", "Nowak", 1995)
```

W poniższym wywołaniu jako parametr `self` przekazana będzie zmienna `jan`. Jako parametr `druga_osoba` przekazana będzie zmienna `anna`.

In [63]:
```
jan.starszy_od(anna)
```

Out[63]: False

Pola i metody określane są wspólną nazwą: **składowe** (ang. *members*) klasy.

Metody `__repr__` i `__str__`

Oprócz metody `__init__` zazwyczaj definiuje się jeszcze metody specjalne odpowiadające za konwersję obiektów klasy na napis. Metodami takimi są

`__repr__` i `__str__`.

Metoda `__str__` jest „dla użytkownika programu” tzn. powinna konwertować obiekt na napis zrozumiały i czytelny dla użytkownika. Np. jeżeli obiekt reprezentuje datę, to powinno być jasne jaka to jest data, a nie np. jak obiekt został utworzony w programie.

Metoda `__repr__` jest „dla programisty” tzn. służy do zamiany obiektu na napis, który umożliwia sprawdzenie wartości obiektu. Metoda `__repr__` jest używana np. przez funkcję wbudowaną `repr`. Jest też używana przez interpreter w trybie interaktywnym do wyświetlania wartości wyrażeń.

Zobaczmy jej działanie na przykładzie typu `Fraction` z modułu `fractions` (typ do operacji na ułamkach):

In [64]:
```
from fractions import Fraction
```

In [65]:
```
trzy_czwarte = Fraction(3, 4)
```

In [66]:
```
repr(trzy_czwarte)
```

Out[66]:
```
'Fraction(3, 4)'
```

Metoda `__repr__` zostanie też użyta do wyświetlenia wartości w trybie interaktywnym:

In [67]:
```
trzy_czwarte
```

Out[67]:
```
Fraction(3, 4)
```

Metoda `__str__` jest między innymi używana przez funkcje `str`, `print` i napisy formatujące:

In [68]:
```
str(trzy_czwarte)
```

Out[68]:
```
'3/4'
```

In [69]:
```
print(trzy_czwarte)
```

```
3/4
```

In [70]:
```
f"{trzy_czwarte}"
```

Out[70]:
```
'3/4'
```

Metody specjalne można również normalnie wywołać, ale rzadko jest to używane:

In [71]:
```
trzy_czwarte.__repr__()
```

Out[71]:
```
'Fraction(3, 4)'
```

In [72]:
```
trzy_czwarte.__str__()
```

Out[72]:
```
'3/4'
```

Konwersja na napis obiektów klasy, w której nie zostały zdefiniowane odpowiednie metody wygląda tak:

In [73]:
```
jan
```

Out[73]:
```
<__main__.Osoba at 0x79fa867f5fd0>
```

In [74]:
```
repr(jan)
```

Out[74]:
```
'<__main__.Osoba object at 0x79fa867f5fd0>'
```

In [75]:
```
print(jan)
```

```
<__main__.Osoba object at 0x79fa867f5fd0>
```

In [76]:
```
str(jan)
```

Out[76]:
```
'<__main__.Osoba object at 0x79fa867f5fd0>'
```

Zdefiniujmy odpowiednie metody `__repr__` i `__str__` w klasie `Osoba`:

In [77]:
```
class Osoba:
    def __init__(self, imie, nazwisko, rok_urodzenia):
        self.imie = imie
        self.nazwisko = nazwisko
        self.rok_urodzenia = rok_urodzenia

    def wiek(self, rok):
        return rok - self.rok_urodzenia

    def starszy_od(self, druga_osoba):
        return self.rok_urodzenia < druga_osoba.rok_urodzenia

    def __repr__(self):
        return (
            f"Osoba({self.imie!r}, {self.nazwisko!r}, "
            f"{self.rok_urodzenia!r})"
        )

    def __str__(self):
        return (
            f"{self.imie} {self.nazwisko} "
            f"urodzony w roku {self.rok_urodzenia}"
        )
```

In [78]:
```
jan = Osoba("Jan", "Kowalski", 2000)
```

In [79]:
```
repr(jan)
```

Out[79]:
```
"Osoba('Jan', 'Kowalski', 2000)"
```

In [80]:
```
jan
```

Out[80]:
```
Osoba('Jan', 'Kowalski', 2000)
```

In [81]:
```
print(jan)
```

```
Jan Kowalski urodzony w roku 2000
```

In [82]:
```
str(jan)
```

Out[82]:
```
'Jan Kowalski urodzony w roku 2000'
```

Metoda `__repr__` powinna w miarę możliwości zwrócić napis w takiej postaci, że wpisanie go w trybie interaktywnym (lub w kodzie źródłowym programu) powinno spowodować utworzenie odpowiedniego obiektu.

Zapis `!r` użyty w napisie formatującym oznacza, że wyrażenie poprzedzające go jest przekształcane na napis za pomocą funkcji `repr` a nie `str`. Porównaj:

In [83]:
```
imie = "Jan"
```

In [84]:
```
print(f"{imie}")
```

```
Jan
```

In [85]:
```
print(f"{imie!r}")
```

```
'Jan'
```

In [86]:
```
print(f"{jan}")
```

```
Jan Kowalski urodzony w roku 2000
```

In [87]:
```
print(f"{jan!r}")
```

```
Osoba('Jan', 'Kowalski', 2000)
```